



Тестування Devops – найкращі практики

Вирішіть проблему розриву між QA та розробниками

Контроль якості має вирішальне значення для процесу випуску програмного забезпечення. Тож чому про це часто думають в останню чергу, коли організація починає ініціативу трансформації DevOps? Існує поширене хибне уявлення про роль контролю якості в середовищі DevOps: що за умови безперервної інтеграції та безперервної доставки всі зусилля щодо якості також можна автоматизувати.

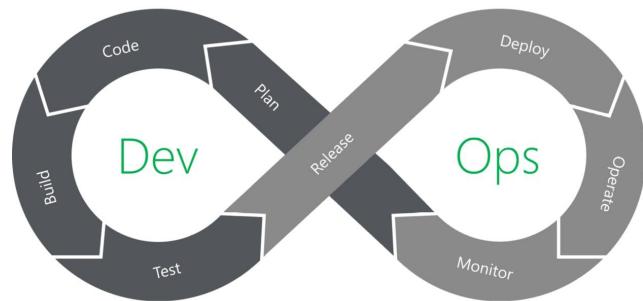
Хоча це правда, що організації досягають вищого рівня автоматизації тестування, ніж будь-коли раніше, готовність до випуску – це бізнес-рішення, а не технічне рішення. І хоча автоматизація може дати вам високий рівень впевненості в тому, що випуск готовий для клієнтів, хороші результати тестування не слід плутати з готовністю до випуску. Отже, як вирішити проблему роз'єднання та забезпечити якість, вбудовану в процеси DevOps? Включіть тестувальників у свою інтегровану команду доставки.



Ітераційне застосування практик та інструментів для кожного етапу DevOps

Замість підходу до інструментарію за принципом копіювання та заміни можна – і навіть бажано – ітеративно застосовувати практики та інструменти для кожного етапу DevOps. Для початку лідери повинні спершу звернути увагу на найбільші проблемні області. Це набагато доступніший метод, оскільки він дозволяє командам застосовувати інструменти та практики з часом, у міру того, як їхні процеси дозрівають і потреби розвиваються.

Згідно з нещодавнім дослідженням Gartner**, як лідери додатків, ви повинні стимулювати свої команди DevOps до розробки своїх методів і інструментів ітеративно, визначаючи найбільші обмеження в їхніх процесах і вибираючи якісні інструменти, які найкраще допомагають їм оптимізувати це обмеження.

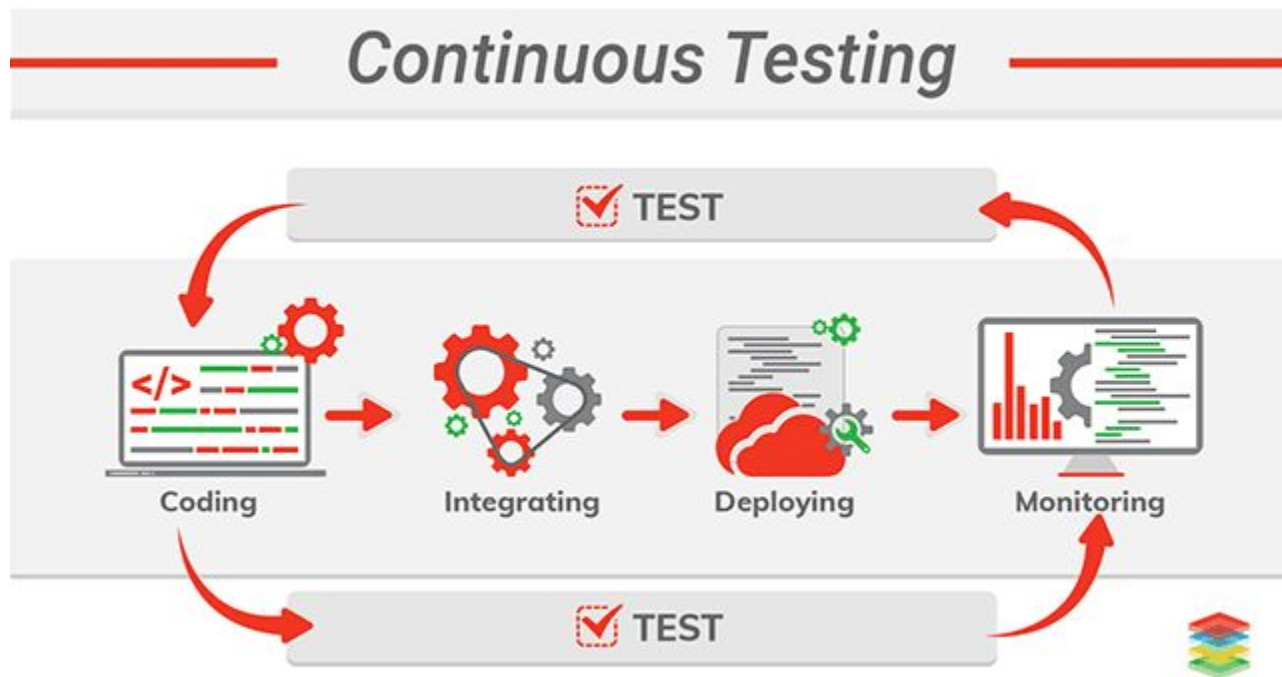


Застосуйте практику безперервного тестування

Нещодавнє дослідження показало, що організації з успішними ініціативами Agile та DevOps роблять ряд речей інакше, ніж їхні аналоги, зокрема активно намагаються перетворити свої практики тестування програмного забезпечення на практики постійного тестування.

Дослідження, проведене під керівництвом аналітика Forrester Дієго Ло Джудіче та на замовлення Tricentis, виявило, що ці організації застосовують п'ять основних практик безперервного тестування: розподіл належних бюджетів на тестування та зосередження на вдосконаленні навичок; здійснення безперервного тестування для задоволення вимог частоти випуску; включаючи автоматизацію наскрізного функціонального тестування; інтеграція тестувальників у міжфункціональні команди та практика тестування зсувом вліво.

Застосуйте практику безперервного тестування



Автоматизуйте тестування там, де це має сенс

Може бути важко визначити ідеальний рівень автоматизації тестування, якого ваша команда має прагнути під час переходу на DevOps. Але оскільки релізи відбуваються значно частіше, обсяг тестування також має значно збільшитися. Ось тут і починає діяти автоматизоване тестування. Згідно з останніми дослідженнями, успішні магазини DevOps значно частіше матимуть високий рівень автоматизації для ключових процесів тестування та контролю якості (дизайн тестових випадків, автоматизація функціональних тестів, керування тестовими даними тощо).



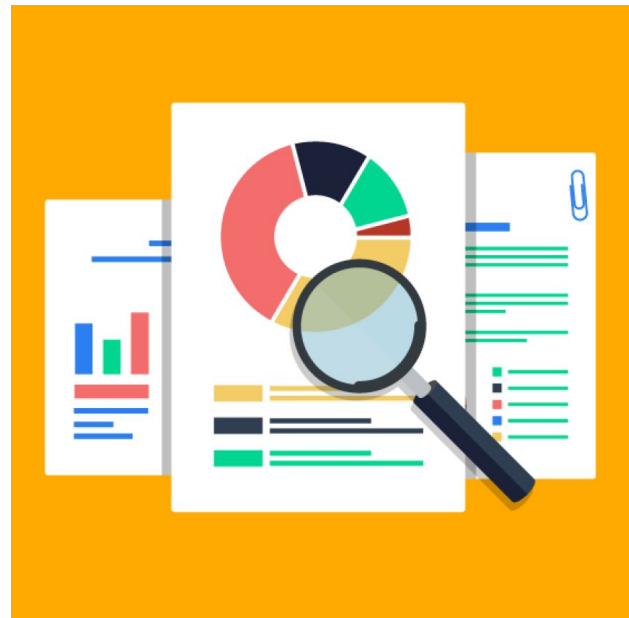
Автоматизуйте тестування там, де це має сенс

Однак командам не слід просто прагнути до максимальної автоматизації. Натомість мета полягає в тому, щоб розробити стратегію автоматизації тестування, яка максимізує ресурси та усуває потребу тестувальникам вручну виконувати повторювані тести. Усуваючи повторні завдання, автоматизація дозволяє тестувальникам витратити свій час більш стратегічно – плануючи й реалізуючи стратегії автоматизації, а також виконуючи пошукове тестування, яке неможливо замінити автоматизацією.

Зосередьтеся на правильних показниках

Визначення тестів, які найбільш доцільно автоматизувати, зводиться до наявності якісних тестових даних. Централізація показників контролю якості, щоб отримати цілісне уявлення про всі ваші тести — у всіх конвеєрах доставки — допоможе вам побачити, як часто тести зазнають невдач, скільки часу потрібно для їх виконання та скільки часу ви витрачаєте на налаштування середовища та дані створення.

Коли ви досягнете відносно високого рівня автоматизації, здатність швидко переглядати й аналізувати показники стане набагато важливішою. Доступ до потрібних показників дозволить вам отримати інформацію, яка допоможе вам зосередити стратегії автоматизації тестування на сферах, у яких це може мати найбільший вплив на ваш бізнес. Отже, які правильні показники тестування? За даними Forrester, експерти Agile та DevOps постійно відносять показники, пов'язані з ризиком, до трійки найцінніших показників. Організації зі зрілою гнучкою практикою та DevOps також зазвичай зосереджуються на контекстних показниках, таких як охоплення вимог, а не просто на підрахунку кількості проведених

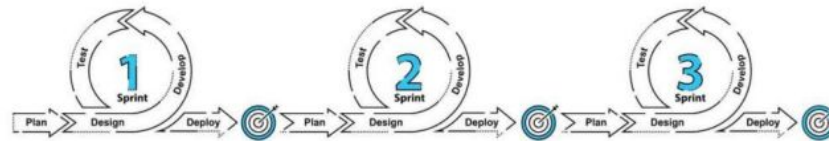


Впровадьте гнучку методологію

Методологію можна визначити як набір найкращих практик, які спрямовані на інтерпретацію та вирішення різних проблем у певній сфері. У розробці програмного забезпечення методологія покликана допомогти покращити процес розробки. Необхідно створити структуру та допомогти спланувати й керувати процесом.

Щоб забезпечити безперебійну роботу процесу розробки програмного забезпечення, важливо мати дорожню карту з найкращою стратегією створення високоякісного програмного забезпечення, яке зручно підтримувати.

До впровадження DevOps існувало кілька методологій, які пропонувалися для структурування процесу розробки: методологія водоспаду, методологія V-моделі, ітераційні та інкрементальні методи тощо. У цих методологіях специфікації більшу частину часу заморожені, не залишаючи місця для змін протягом усього процесу розробки. Крім того, вони часто не дозволяють часті доставки. У моделі водоспаду, наприклад, продукт приходить до користувачів лише після завершення всього процесу, що означає, що один крок за і потребами користувачів не відповідає.



AGILE METHODOLOGY

Впровадьте гнучку методологію

Гнучка розробка дозволяє та сприяє безперервній інтеграції, що відповідає культурі DevOps. Завдяки Agile та його реалізації розробка програмного забезпечення розбивається на невеликі завдання, які називаються історіями. Ітеративний і поступовий підхід (наприклад, Scrum) дає можливість постійної інтеграції. Щоразу, коли історія закінчена, вона надсилається в спільне сховище, де вона тестується та підтверджується спочатку як окрема функція, а потім як частина системи. Після перевірки продукту помилки повідомляються в резервний журнал і будуть частиною наступної ітерації. Власник продукту, призначений на початку проекту, братиме участь у процесі розробки: UAT, перевірка вимог, пріоритетність невиконаних завдань тощо.

Agile допомагає підійти до складної проблеми, розбиваючи її на більш керовані частини функцій і модулів. Ітерація та хороший контроль за часом гарантують, що продукт розробляється відповідно до вимог і потреб клієнтів, що призводить до вищої якості продукту, що є однією з цілей DevOps.

Впроваджуйте невеликі ітерації для кращого контролю

Невеликі зміни – це правильний шлях до безперервної інтеграції. Введення кількох змін за раз дозволяє часту інтеграцію. Після внесення невеликих змін і успішної інтеграції клієнту надається новий випуск. На цьому етапі, якщо виникнуть помилки, буде легше виявити їх основні причини. Ці помилки часто виявляються на ранній стадії, що призводить до меншої кількості проблем у виробничому середовищі. Перевірку та розгортання можна виконати дуже швидко, а також скоротити час виходу на ринок, а також часту доставку.

Not like this...



...instead like this!



Документуйте процес

Документація також важлива для процесу розгортання. Розгортання дещо відрізняється від однієї програми до іншої. Ці відмінності можуть включати: конфігурацію, необхідну для програми (папка завантаження, база даних тощо), і передумови для запуску програмного забезпечення (ЦП, пам'ять тощо). Стандартизовані документи можуть допомогти подолати цю перешкоду. Процес розгортання має бути зрозумілим, щоб ми не витрачали час на звітування про проблеми з розгортанням. Ми повинні пам'ятати, що розгортання в контексті DevOps відбуватиметься часто, тому документація має бути якомога детальнішою, щоб пришвидшити процес

Документуйте процес

Підсумовуючи, є чотири типи документації, які є важливими під час впровадження DevOps:

- Документація вихідного коду
- Проектно-технічна документація
- Як розгорнути
- Інструкції для кінцевих користувачів



Виберіть правильні інструменти

Кожна найкраща практика має підтримуватися інструментами. Ці інструменти мають бути стандартизовані серед розробників програмного забезпечення та команди операцій. Ця стандартизація особливо важлива, коли процес включає більше ніж одного користувача.

Ось кілька типів інструментів, які я рекомендую:

- Комунікаційні засоби
- Інструменти безперервного тестування
- Інструменти контролю версій
- Інструменти моніторингу та налаштування
- Гнучкі інструменти



Будьте реактивними, проактивними та відкритими до відгуків

Для високоякісного програмного забезпечення важливо бути відкритим до відгуків і діяти відповідно до них, як тільки вони надходять.

Зворотний зв'язок можна отримати трьома різними каналами:

- Від операційної команди до команди розробників і навпаки
- Від кінцевих користувачів до команди розробників
- З журналу програми: дозвольте програмному забезпеченню, яке ми розробляємо, надсилати нам відгуки

Операційна команда надає відгук, повідомляючи про помилки чи вади або пропонуючи вдосконалення. Кілька інструментів налаштування та моніторингу, як-от Munin або Sentry One, тепер використовуються для відстеження проблем, вони стали настільки ефективними, що швидко відстежувати проблеми та виявляти потенційні вузькі місця стало легше. Пропозиції та повідомлення про помилки допомагають покращити якість коду та навіть допомагають передбачити проблеми та знайти рішення до того, як вони вплинуть на роботу. Виправлення повідомленої проблеми, як тільки вона виникає, є більш ніж хорошою практикою. Слід докласти зусиль, щоб скоротити час на виправлення помилки.

Будьте реактивними, проактивними та відкритими до відгуків

Застосування уроків, отриманих з попередніх викликів, є одним із способів бути більш проактивним. Під час етапу перевірки можуть виникати ситуації, які можуть спричинити проблеми, програмне забезпечення має успішно пройти ці ситуації, щоб отримати право на робоче розгортання.

Ще один спосіб бути більш проактивним – це відкрите спілкування: менше припускайте та більше інформуйте. Усі ми хоч раз страждали від поганого спілкування. Команда розробників іноді може займатися налагодженням протягом кількох днів, щоб зрозуміти, що це сталося через зміни в конфігурації робочого середовища.



Висновок

Застосувати ці прийоми не так складно, як може здатися. Коли організація налаштована з потрібними інструментами та зрозуміла культура, простіше застосовувати принципи DevOps щодня. Спілкування та співпраця за допомогою правильних інструментів забезпечують безперебійну роботу процесу розробки програмного забезпечення.

Зрештою, DevOps має виникнути природно, це так само природно, як і зміни, які постійно відбуваються в організації та бізнесі.



pytest

Основи pytest

Що таке pytest?

- pytest — це потужний фреймворк для тестування, спеціально створений для Python, який спрощує процес тестування.
- У своїй основі pytest пропонує чистий та простий синтаксис і потужні функції, що дозволяє розробникам писати короткі та легко підтримувані тести.

Чому варто використовувати pytest для тестування Python?

- pytest пропонує численні переваги, зокрема простоту та гнучкість, що дозволяє розробникам ефективно писати тести.
- Інтуїтивний синтаксис та широкий екосистем плагінів підвищують продуктивність і дозволяють створювати тестові рішення, адаптовані під конкретні потреби.
- Крім того, pytest легко інтегрується з існуючими інструментами та фреймворками, розширюючи його корисність у різноманітних проектах на Python.

Встановлення pytest

Вимоги:

Python: Перевірте, що Python встановлений на вашій системі. pytest підтримує версії 3.6 і вище.

Встановлення за допомогою віртуального середовища:

1. **Створення віртуального середовища:**
Створіть віртуальне середовище за допомогою інструменту, такого як `virtualenv` або `venv`, щоб ізолювати залежності вашого проекту.
2. **Активація віртуального середовища:**
Активуйте віртуальне середовище, використовуючи відповідну команду для вашої операційної системи:
`source venv/bin/activate`
3. **Встановлення pytest:**
У активованому віртуальному середовищі виконайте команду:
`pip install pytest`
4. **Деактивація віртуального середовища (опційно):**
Після встановлення pytest можна деактивувати віртуальне середовище командою:
`deactivate`

Структура базового тесту

Базовий тест у pytest розміщується всередині функції Python.

Назва функції зазвичай починається з `test_`, що слугує маркером для pytest для її розпізнавання як тесту.

Усередині цієї функції визначаються **assert-вирази** (за допомогою ключового слова `assert` у Python) для перевірки конкретної поведінки вашого коду.

```
def add_numbers(x, y):  
    return x + y
```

```
def test_addition():  
    assert add_numbers(2, 3) == 5
```

Асепти (Assertions) у pytest

Асепти відіграють ключову роль у перевірці поведінки коду під час тестування. Розгляньмо, як працюють асепти у pytest.

У pytest доступні різні поширені асепти, що дозволяють проводити всебічне тестування. Вони дають можливість розробникам перевіряти очікувані результати та впевнено забезпечувати правильність роботи коду.

Ефективне використання ключового слова `assert` є важливим для написання надійних тестів. Стратегічне розміщення `assert` дозволяє перевіряти конкретні умови, обробляти крайні випадки та підвищувати покриття тестами. Розуміння того, коли і як застосовувати `assert`, допомагає розробникам створювати комплексні тестові набори, які повністю перевіряють функціональність коду.

Запуск тестів у pytest

pytest надає широкий набір параметрів командного рядка, які відповідають різним потребам тестування. Ці параметри дозволяють детально контролювати процес виконання тестів, даючи можливість розробникам налаштовувати тестове середовище відповідно до своїх потреб. Наприклад, параметр `-v` підвищує детальність виводу, надаючи розширену інформацію про результати тестів та порядок їх виконання. Параметр `--tb` дозволяє задати рівень деталізації відображення traceback, що полегшує ефективне налагодження. Використовуючи ці параметри командного рядка, розробники можуть оптимізувати процес тестування та отримувати цінну інформацію по поведінку свого коду.

pytest дозволяє точно націлюватися на окремі тести або підмножини тестів. Вказуючи назви тестів, каталоги або маркери, розробники можуть фокусуватися на конкретних ділянках коду. Такий цільовий підхід не лише пришвидшує виконання тестів, але й сприяє ретельному тестуванню критично важливих функціональностей, забезпечуючи цілісність і надійність коду.

Фікстури (Fixtures) у pytest

У pytest фікстури — це спеціалізовані функції, призначені для надання тестових даних або налаштування необхідних ресурсів перед виконанням тесту. Вони інкапсулюють загальні дії підготовки та завершення (setup і teardown), сприяючи повторному використанню коду та підвищуючи підтримуваність тестів. Фікстури можна використовувати для ініціалізації підключень до бази даних, налаштування об'єктів-моків або підготовки будь-яких інших ресурсів, необхідних для тестування.

Фікстури забезпечують послідовне тестове середовище, виконуючи дії підготовки перед кожним тестом і завершення після його виконання. До запуску тесту фікстура готує необхідний контекст або ресурси. Після завершення тесту фікстура очищує виділені ресурси або виконує необхідні дії teardown. Такий підхід гарантує, що кожен тест працює незалежно та надійно, мінімізуючи побічні ефекти та забезпечуючи відтворюваність результатів. Використовуючи фікстури, розробники можуть оптимізувати процес тестування та зосередитися на перевірці поведінки свого коду з упевненістю.

```
def calculate_rectangle_area(width, height):  
    return width * height
```

```
import pytest  
  
@pytest.fixture  
def rectangle_dimensions():  
    width = 5  
    height = 10  
    return width, height  
  
def test_calculate_area(rectangle_dimensions):  
    width, height = rectangle_dimensions  
    area = calculate_rectangle_area(width, height)  
    assert area == 50
```


Параметризовані тести (Parameterized Tests) у pytest

Параметризовані тести — це корисна функція pytest, яка дозволяє виконувати одну й ту саму тестову логіку з різними вхідними значеннями. Розглянемо концепцію параметризації та її практичне застосування у тестуванні.

Параметризація дозволяє запускати тести з різними наборами вхідних даних або конфігурацій, що спрощує перевірку поведінки функції в різних сценаріях.

У pytest параметризовані тести реалізуються за допомогою декоратора `@pytest.mark.parametrize`, де вказуються параметри та їхні відповідні значення. Це дозволяє розробникам виконувати одну й ту саму тестову логіку з різними вхідними даними, забезпечуючи всебічне покриття тестами та підвищуючи ефективність тестування.

```
def is_even(number):  
    return number % 2 == 0
```

```
import pytest  
  
@pytest.mark.parametrize("number, expected_result", [  
    (2, True),  
    (3, False),  
    (10, True),  
    (15, False)  
])  
  
def test_is_even(number, expected_result):  
    assert is_even(number) == expected_result
```

Використання плагінів у pytest

Плагіни pytest безшовно інтегруються з фреймворком pytest, розширюючи його функціональність без необхідності значних змін у наявному коді тестів. Вони надають **хуки та API**, що дозволяють розробникам підвищувати автоматизацію тестування, покращувати покриття тестами та оптимізувати робочі процеси тестування з мінімальними зусиллями.

Просте встановлення та налаштування плагінів відкриває додаткові функції та оптимізації, адаптовані до конкретних потреб тестування. Більше того, плагіни зазвичай дотримуються філософії pytest — простоти та гнучкості, що забезпечує сумісність з існуючими конвенціями та робочими процесами pytest.

Приклади популярних плагінів:

- `pytest-cov` — для аналізу покриття коду тестами
- `pytest-xdist` — для паралельного виконання тестів
- `pytest-html` — для генерації HTML-звітів
- `pytest-django` — для тестування Django-застосунків
- `pytest-flask` — для тестування Flask-застосунків
- `pytest-mock` — для спрощеного використання моків та патчів

Маркери (Markers) у pytest

Pytest дозволяє розробникам створювати власні маркери за допомогою декоратора `@pytest.mark`. Ці маркери виступають як анотації, які можна застосовувати до окремих тестових функцій або цілих тестових модулів. Наприклад, можна створити маркери `@pytest.mark.smoke` для smoke-тестів або `@pytest.mark.performance` для тестів продуктивності. Маркери допомагають категоризувати тести за їхніми характеристиками або вимогами, що полегшує цільове виконання тестів.

Після створення власних маркерів pytest надає зручний спосіб запускати тести, пов'язані з конкретними маркерами, за допомогою параметра `-m`, після якого вказується назва маркера. Наприклад, щоб виконати лише smoke-тести, можна запустити:

```
pytest -m smoke
```

Такий селективний підхід до виконання тестів дозволяє розробникам зосередитися на певних наборах тестів або сценаріях, підвищуючи ефективність тестування та скорочуючи час отримання зворотного зв'язку. Крім того, pytest підтримує логічні комбінації маркерів, що дозволяє створювати ще більш складні критерії відбору тестів.

Налаштування з pytest.ini

Файл `pytest.ini` дозволяє розробникам налаштовувати різні аспекти виконання тестів та їх поведінку. Наприклад:

- Вказувати **шляхи до тестів**, щоб контролювати, які каталоги pytest сканує для пошуку тестів.
- Створювати **власні маркери** для категоризації тестів за такими ознаками, як важливість чи тип.
- Налаштовувати **рівень детальності виводу** тестів (verbosity) та формати звітів, наприклад, звичайний текст або HTML.
- Налаштовувати **опції логування** для тестів.

Завдяки налаштуванню `pytest.ini` розробники можуть встановлювати **послідовне тестове середовище** для різних проектів, оптимізувати робочі процеси тестування та забезпечувати дотримання специфічних для проекту конвенцій тестування.

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
markers =
    smoke: mark a test as a smoke test
    regression: mark a test as a regression test
    integration: mark a test as an integration test
    slow: mark a test as a slow-running test
addopts = -v -rsxX --tb=short
log_cli = true
log_cli_level = INFO
log_file = pytest.log
console_output_style = classic
```

Покриття тестами (Test Coverage)

Покриття тестами вимірює, наскільки вихідний код охоплений тестами. Воно показує відсоток рядків коду, гілок або функцій, які були виконані під час тестування, порівняно з усім кодовим базисом.

Існують різні інструменти для створення звітів про покриття у проєктах на Python, наприклад, [coverage.py](#), [pytest-cov](#) та [codecov](#). Ці інструменти аналізують кодову базу та надають детальні звіти, які показують, які частини коду охоплені тестами, а які ні, допомагаючи розробникам оцінити повноту тестування та визначити області для покращення.

```
===== 5 passed in 0.28s =====
(venv) [roman@MBP-de-Roman /Volumes/Projects/DanIT/devops3/Lessons/Testing-1]
| => pytest --cov=app
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.1.1, pluggy-1.4.0
rootdir: /Volumes/Projects/DanIT/devops3/Lessons/Testing-1
plugins: cov-5.0.0, flask-1.3.0
collected 5 items

test_app.py .... [100%]

----- coverage: platform darwin, python 3.12.2-final-0 -----
Name      Stmts   Miss  Cover
-----
app.py      73     10    86%
TOTAL      73     10    86%
```

Розширені можливості pytest

pytest надає багатий набір **хуків (hooks)**, що дозволяє розробникам налаштовувати різні аспекти процесу тестування. Хуки дають змогу втручатися на різних етапах виконання тестів, наприклад, до та після запуску тестів, що забезпечує розширену логіку підготовки, завершення та налаштування тестів.

pytest підтримує **умовне виконання тестів**, дозволяючи запускати тести залежно від динамічних умов або зовнішніх факторів. Використовуючи маркери, параметри командного рядка або власну логіку всередині тестових функцій, розробники можуть вибірково виконувати тести за певними критеріями, підвищуючи гнучкість і ефективність тестування.

```
import pytest
import sys

def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True

@pytest.mark.skipif(sys.version_info < (3, 8), reason="Requires Python 3.8 or higher")
@pytest.mark.parametrize("number, expected_result", [
    (2, True),
    (3, True),
    (4, False),
    (5, True),
])

def test_is_prime(number, expected_result):
    assert is_prime(number) == expected_result
```

Налаштування тестового середовища Flask

При налаштуванні тестового середовища Flask розробники зазвичай створюють окрему конфігурацію, спеціально адаптовану для тестування. Ця конфігурація часто наслідує основну конфігурацію застосунку, але може змінювати певні налаштування відповідно до потреб тестових сценаріїв. Використання тестової конфігурації дозволяє забезпечити ізолюваність тестів і послідовну поведінку під час їх виконання.

Для взаємодії з Flask-застосунком під час тестування використовується **тестовий клієнт Flask**. Цей клієнт надає симульований інтерфейс для виконання HTTP-запитів до ендпоінтів застосунку без необхідності запуску повного HTTP-сервера. Розробники можуть надсилати різні типи запитів, такі як **GET**, **POST**, **PUT** та **DELETE**, до різних маршрутів Flask-застосунку. Після цього вони можуть аналізувати отримані відповіді та робити асerti для перевірки того, що застосунок поводить ся очікувано за різних умов. Це дозволяє проводити всебічне тестування функціональності застосунку у контрольованому середовищі.

Написання тестів для маршрутів Flask

При написанні тестів для маршрутів Flask розробники зазвичай зосереджуються на перевірці поведінки окремих ендпоінтів застосунку. Це передбачає створення тестів для симуляції різних HTTP-запитів, таких як **GET**, **POST**, **PUT** та **DELETE**, до різних маршрутів застосунку.

Під час тестування розробники ретельно аналізують відповіді, отримані від цих маршрутів, щоб переконатися, що вони відповідають очікуваним критеріям. Це включає перевірку **статус-кодів**, **заголовків відповіді** та **вмісту тіла відповіді**. Крім того, розробники можуть перевіряти поведінку конкретних функціональностей, реалізованих у цих маршрутах, таких як аутентифікація, перевірка введених даних або обробка даних.

```
# app.py

from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/')
def hello():
    return jsonify(message='Hello, Flask!'), 200
```

```
# test_app.py

import pytest
from app import app

@pytest.fixture
def client():
    with app.test_client() as client:
        yield client

def test_hello_route(client):
    response = client.get('/')
    assert response.status_code == 200
    assert response.json == {'message': 'Hello, Flask!'}
```


Мокінг у тестах Flask

Мокінг — це важлива техніка при тестуванні Flask, оскільки вона дозволяє розробникам **симулювати поведінку зовнішніх залежностей або сервісів** без фактичного їх виклику.

Це особливо корисно при тестуванні Flask-застосунків, які взаємодіють із базами даних, API або іншими зовнішніми системами.

```
# app.py
import requests
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/weather')
def get_weather():
    try:
        response = requests.get('https://api.openweathermap.org/data/2.5/weather?q=Kyiv&appid=your_api_key')
        response.raise_for_status()
        weather_data = response.json()
        return jsonify(weather_data), 200
    except requests.RequestException:
        return jsonify({'error': 'Failed to fetch weather data'}), 500
```

```
# test_app.py
import pytest
from app import app
from unittest.mock import patch, Mock

@pytest.fixture
def client():
    with app.test_client() as client:
        yield client

@patch('app.requests.get')
def test_weather_success(mock_get, client):
    mock_response = Mock()
    mock_response.raise_for_status = Mock()
    mock_response.json.return_value = {'temperature': 20, 'conditions': 'Sunny'}
    mock_get.return_value = mock_response

    response = client.get('/weather')

    assert response.status_code == 200
    assert response.json == {'temperature': 20, 'conditions': 'Sunny'}

@patch('app.requests.get')
def test_weather_failure(mock_get, client):
    mock_get.side_effect = requests.RequestException()

    response = client.get('/weather')

    assert response.status_code == 500
    assert response.json == {'error': 'Failed to fetch weather data'}
```

Розширені сценарії тестування Flask

Механізми аутентифікації та авторизації є критично важливими компонентами веб-застосунків для забезпечення безпечного доступу до ресурсів. Тестування цих механізмів передбачає **симуляцію аутентифікації користувача** та перевірку того, що захищені маршрути доступні лише авторизованим користувачам. Це може включати мокінг сесій користувачів, генерацію токенів аутентифікації або використання власних заголовків авторизації у тестових запитах.

Управління сесіями є важливим для підтримки стану користувача між запитами у веб-застосунках. Тестування функціоналу, пов'язаного з сесіями, включає роботу з даними сесії, встановлення cookie сесії та перевірку, що функції, залежні від сесії, працюють належним чином. Розробники можуть використовувати техніки, такі як мокінг об'єктів сесії, маніпуляції з cookie сесії або використання утиліт тестування Flask для симуляції поведінки сесій під час тестів.

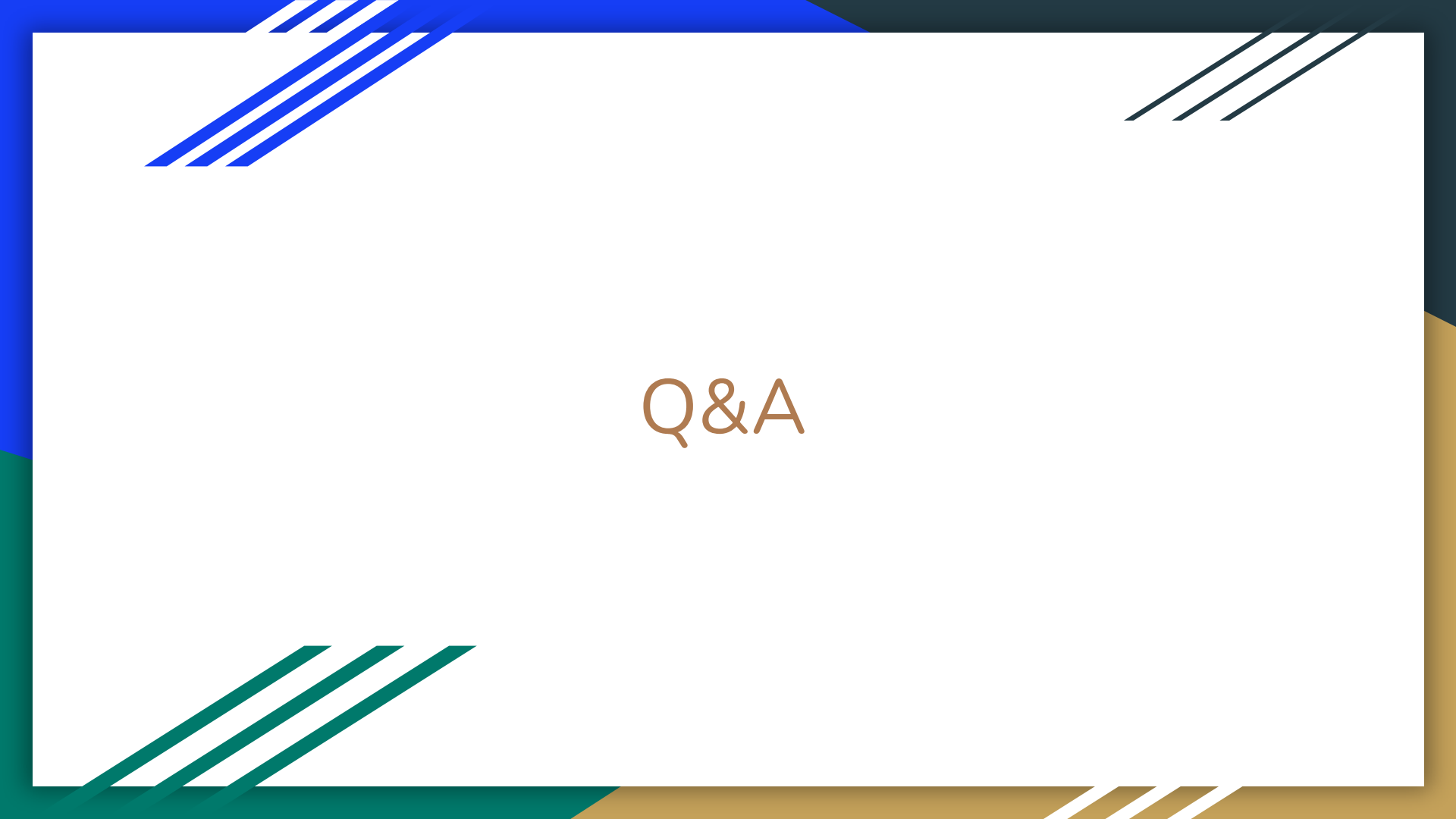
Кращі практики у pytest

Організація файлів тестів:

- Групуйте тести у логічні модулі або пакети відповідно до функціональності або компонентів застосунку.
- Використовуйте директорії для ієрархічної організації тестів, відображаючи структуру самого застосунку.
- Зберігайте файли тестів окремо від вихідного коду, щоб підтримувати чисту структуру проекту.

Конвенції найменування тестів:

- Використовуйте описові назви, які передають мету та поведінку, що тестується.
- Префіксуйте функції тестів словом `test_`, щоб pytest міг їх виявляти.
- Дотримуйтеся послідовної конвенції найменування файлів і функцій тестів у всьому тестовому наборі.



Q&A